

Table of Contents

ИНСТРУКЦИЯ ПО ВНЕДРЕНИЮ

Minimal Patch Set — Kernel Isolation Repair (v0.1)

Проект: ENIGMA v0.5.3.1.9 **Документ:** Инструкция по созданию и внедрению патча **Тип:** Техническое задание на миграцию (не код внедрения, а документация-инструкция) **Цель патча:** закрыть 5 RNG leaks + 3 non-seeded DecisionHub instantiation, создать единый deterministic kernel context

Принцип патча: без изменения архитектуры. Physics layer НЕ трогаем концептуально. Только kernel isolation repair.

0. Контекст: что патч делает и почему

0.1. Текущее состояние (ДО патча)

ENIGMA сейчас — **kernel-incomplete deterministic world engine**:

-  Kernel infrastructure существует (TemporalEngine, seed support в DecisionHub.__init__(seed=...), _deterministic_jitter pattern в event_compiler.py)
-  Kernel isolation broken (5 пробоев детерминизма)
-  Physics overlay не формализован (reconcile_state смешивает seconds→ticks)
-  CK-style systems отсутствуют (нет lineage, aging, WorldChronicle)

Подтверждение из SUPERBOX drift report

(дрейф_mass_traversal.md): - Дрейф D (Каузальный): 140/215 сравнений = **65% causal drift** - ФАЗА 3 (100k сравнений): НЕ ПРОЙДЕН  - Структурный дрейф: ОБНАРУЖЕН 

0.2. Что патч гарантирует (ПОСЛЕ применения)

Свойство	До	После
Same input → same L1		
Same tick sequence → same decisions		

Свойство	До	После
Hidden randomness	5 мест	0
Replayable simulation core	✗	✓
SUPERBOX causal drift	65%	~0-2%

0.3. Что патч НЕ решает (намеренно)

- Physics overlay (seconds) — не трогаем
- reconcile_state — остаётся conceptual boundary
- WorldChronicle — не реализован
- Lineage — не реализован
- Aging — не реализован

Это **additive migration**, не rewrite. После применения можно строить CK-style systems поверх deterministic kernel.

1. Порядок применения (обязательный)

Патч — **атомарная миграция** из 7 файлов + 1 validation hook.

Применять строго в указанном порядке, потому что есть зависимости:

STEP 1 → kernel_rng.py (NEW — foundation, без него остальные не работают)
STEP 2 → tick_orchestrator.py (MODIFY — расширить _TickContext полем rng)
STEP 3 → decision_hub.py (MODIFY — adapter layer, принять rng параметр)
STEP 4 → npc_tick_pipeline.py (MODIFY — передать rng в DecisionHub)
STEP 5 → life_engine.py (MODIFY — 2 fix: DecisionHub + random.* replacement)
STEP 6 → movement_engine.py (MODIFY — random.uniform → rng.uniform)
STEP 7 → state_applicator.py (MODIFY — random.choices → rng.choices)
STEP 8 → SUPERBOX validation (NEW — determinism test hook)

Зависимости: - STEP 2 требует STEP 1 (нужен KernelRNG класс) - STEP 3 требует STEP 1 (нужен KernelRNG тип) - STEP 4-7 требуют STEP 2 (нужен ctx.rng) - STEP 8 требует STEP 1-7 (валидирует результат)

Откат: если любой шаг падает — откатывать все. Патч атомарный.

2. STEP 1 — backend/app/services/npc/kernel_rng.py (NEW FILE)

2.1. Назначение

Единый deterministic RNG, привязанный к (tick, npc_id). Это **единственный источник случайности** в kernel layer. Все random.* вызовы в kernel должны идти через KernelRNG.

2.2. Создать файл

Путь: backend/app/services/npc/kernel_rng.py

Содержимое:

```
"""
path: backend/app/services/npc/kernel_rng.py
Назначение: Deterministic RNG bound to (tick, npc_id).
            Единственный источник случайности в kernel layer.
Зависимости: stdlib (random, hashlib)
Основные сущности: KernelRNG

АРХИТЕКТУРНЫЙ ПРИНЦИП (Metaphysical Core ADR):
    Tick = causal spine (non-negotiable).
    Kernel never depends from seconds.
    All kernel randomness MUST go through KernelRNG.

ИСПОЛЬЗОВАНИЕ:
    rng = KernelRNG(tick=42, npc_id="maid_lusya")
    if rng.random() < 0.4: # deterministic for (42, "maid_lusya")
    ...

ГАРАНТИИ:
    - Same (tick, npc_id) → same RNG sequence
    - Different npc_id on same tick → independent sequences
    - Replay determinism: same input → same output
"""

import random
import hashlib

class KernelRNG:
    """
    Deterministic RNG bound to (tick, npc_id).

    This is the ONLY randomness source in kernel layer.
    Physics layer (seconds-based) does NOT use this class.

    Seed derivation:
        seed_raw = f"{tick}:{npc_id}".encode()
```

```
seed = int(sha256(seed_raw).hexdigest(), 16)
```

Guarantees:

- Same (tick, npc_id) → same seed → same sequence
- Different npc_id → different seed (independence)
- Cryptographic hash ensures no seed collisions

"""

```
def __init__(self, tick: int, npc_id: str):
    if not isinstance(tick, int) or tick < 0:
        raise ValueError(f"tick must be non-negative int, got {tick}")
    if not isinstance(npc_id, str) or not npc_id:
        raise ValueError(f"npc_id must be non-empty str, got {npc_id!r}")
```

```
seed_raw = f"{tick}:{npc_id}".encode("utf-8")
seed = int(hashlib.sha256(seed_raw).hexdigest(), 16)
```

```
self._rng = random.Random(seed)
self.seed = seed
self.tick = tick
self.npc_id = npc_id
```

— Wrappers (explicit control boundary) —————
Каждый метод явно делегирует в _rng. Это создаёт clear API boundary:
kernel code использует ctx.rng.random(), не random.random().

```
def random(self) -> float:
    """Return [0.0, 1.0) — deterministic for (tick, npc_id)."""
    return self._rng.random()
```

```
def choice(self, seq):
    """Choose random element from seq — deterministic."""
    return self._rng.choice(seq)
```

```
def uniform(self, a: float, b: float) -> float:
    """Return [a, b) — deterministic."""
    return self._rng.uniform(a, b)
```

```
def choices(self, seq, weights=None, k: int = 1):
    """Choose k elements — deterministic."""
    return self._rng.choices(seq, weights=weights, k=k)
```

```
def randint(self, a: int, b: int) -> int:
    """Return [a, b] inclusive — deterministic."""
    return self._rng.randint(a, b)
```

```
def __repr__(self) -> str:
    return f"KernelRNG(tick={self.tick}, npc_id={self.npc_id!r},\nseed={self.seed})"
```

2.3. Регрессионный тест (создать рядом)

Путь: backend/tests/test_kernel_rng.py

```
"""Тесты KernelRNG determinism."""
from app.services.npc.kernel_rng import KernelRNG


def test_same_tick_npc_same_sequence():
    """Same (tick, npc id) → same RNG sequence."""
    rng_a = KernelRNG(tick=42, npc_id="maid_lusya")
    rng_b = KernelRNG(tick=42, npc_id="maid_lusya")

    assert rng_a.seed == rng_b.seed
    assert rng_a.random() == rng_b.random()
    assert rng_a.uniform(-1, 1) == rng_b.uniform(-1, 1)


def test_different_npc_independent():
    """Different npc id → different sequences."""
    rng_a = KernelRNG(tick=42, npc_id="maid_lusya")
    rng_b = KernelRNG(tick=42, npc_id="thief_shadow")

    assert rng_a.seed != rng_b.seed
    # Very unlikely to be equal
    assert rng_a.random() != rng_b.random() or rng_a.uniform(0, 1) !=
rng_b.uniform(0, 1)


def test_different_tick_independent():
    """Different tick → different sequences."""
    rng_a = KernelRNG(tick=41, npc_id="maid_lusya")
    rng_b = KernelRNG(tick=42, npc_id="maid_lusya")

    assert rng_a.seed != rng_b.seed


def test_invalid_inputs():
    """Invalid inputs → ValueError."""
    import pytest
    with pytest.raises(ValueError):
        KernelRNG(tick=-1, npc_id="x")
    with pytest.raises(ValueError):
        KernelRNG(tick=0, npc_id="")
    with pytest.raises(ValueError):
        KernelRNG(tick="not_int", npc_id="x")


def test_choice_deterministic():
```

```

"""choice returns same element for same (tick, npc_id)."""
seq = ["a", "b", "c", "d"]
rng_a = KernelRNG(tick=100, npc_id="guard_borko")
rng_b = KernelRNG(tick=100, npc_id="guard_borko")

assert rng_a.choice(seq) == rng_b.choice(seq)

def test_choices_deterministic():
    """choices with weights returns same result."""
    seq = ["head", "torso", "arm"]
    weights = [1, 3, 2]
    rng_a = KernelRNG(tick=200, npc_id="merchant_goran")
    rng_b = KernelRNG(tick=200, npc_id="merchant_goran")

    assert rng_a.choices(seq, weights=weights, k=1) == rng_b.choices(seq,
weights=weights, k=1)

```

2.4. Критерий приёмки

```
cd backend && python -m pytest tests/test_kernel_rng.py -v
```

Все 6 тестов должны проходить.

3. STEP 2 — backend/app/services/tick_orchestrator.py (MODIFY)

3.1. Назначение

Расширить `_TickContext` полем `rng: KernelRNG`. Без этого `ctx.rng` недоступен в downstream модулях.

3.2. Контекст (что сейчас)

Файл: backend/app/services/tick_orchestrator.py

`_TickContext` определён на строке 157:

```

@dataclass
class _TickContext:
    """Внутренний контекст тика — живёт только внутри execute()."""
    campaign_id: str
    scene_state: dict
    tick_number: int
    drf_bus: DRFBus
    # ... много полей ...

```

Создаётся в двух местах: - строка 469 (idle path) - строка 680 (player path)

3.3. Изменение 1 — добавить поле в `_TickContext`

ЗАМЕНИ (строка 157, после `class _TickContext:`):

```
@dataclass
class _TickContext:
    """Внутренний контекст тика — живёт только внутри execute()."""
    campaign_id: str
    scene_state: dict
    tick_number: int
    # ADR-134: Instance-level bus — обязательно передаётся без дефолта (выше
    # полей с default_factory)
    drf_bus: DRFBus
```

НА:

```
@dataclass
class _TickContext:
    """Внутренний контекст тика — живёт только внутри execute()."""
    campaign_id: str
    scene_state: dict
    tick_number: int
    # ADR-134: Instance-level bus — обязательно передаётся без дефолта (выше
    # полей с default_factory)
    drf_bus: DRFBus
    # KERNEL-ISOLATION: per-tick RNG factory.
    # Создаёт KernelRNG для каждого NPC по запросу (lazy).
    # НЕ хранит RNG state напрямую — хранит factory, чтобы каждый NPC
    # получил независимый deterministic stream.
    rng_factory: Optional[Callable[[str], "KernelRNG"]] = None
```

3.4. Изменение 2 — импорт `KernelRNG`

ДОБАВИТЬ в начало файла (после существующих импортов, примерно строка 30):

```
from app.services.npc.kernel_rng import KernelRNG
from typing import Callable
```

3.5. Изменение 3 — `factory` при создании `ctx`

ЗАМЕНИ в обоих местах создания `_TickContext` (строки 469 и 680):

```
ctx = _TickContext(
    campaign_id=campaign_id,
    scene_state=scene_state,
    tick_number=tick_number,
    drf_bus=drf_bus,
    # ... ОСТАЛЬНЫЕ ПОЛЯ ...
)
```

НА:

```

# KERNEL-ISOLATION: factory для per-NPC deterministic RNG.
# Каждый NPC получает свой RNG, seeded by (tick, npc_id).
_tick_for_rng = tick_number
_rng_factory = lambda npc_id: KernelRNG(tick=_tick_for_rng,
npc_id=npc_id)

ctx = _TickContext(
    campaign_id=campaign_id,
    scene_state=scene_state,
    tick_number=tick_number,
    drf_bus=drf_bus,
    rng_factory=_rng_factory,
    # ... ОСТАЛЬНЫЕ ПОЛЯ ...
)

```

3.6. Изменение 4 — helper метод для доступа к RNG

ДОБАВИТЬ метод в `_TickContext` (после `__init__` или в конец класса):

```

def rng_for(self, npc_id: str) -> KernelRNG:
    """Возвращает deterministic KernelRNG для данного NPC на текущем тике.

    Использование:
        rng = ctx.rng_for("maid_lusya")
        if rng.random() < 0.4:
            ...

    """
    if self.rng_factory is None:
        # Fallback для legacy тестов, где factory не задан.
        # В production этого быть не должно.
        import logging
        logging.getLogger(__name__).warning(
            f"[TICK_CONTEXT] rng_factory is None, creating ad-hoc KernelRNG "
            f"for npc={npc_id} tick={self.tick_number}. "
            f"This indicates incomplete initialization."
        )
        return KernelRNG(tick=self.tick_number, npc_id=npc_id)
    return self.rng_factory(npc_id)

```

3.7. Критерий приёмки

```

cd backend && python -c "
from app.services.tick_orchestrator import TickOrchestrator
# Проверка, что _TickContext принимает rng_factory
import inspect
sig = inspect.signature(TickOrchestrator)
print('TickOrchestrator imports OK')
"

```

Не должно быть ImportError.

4. STEP 3 — backend/app/services/npc/decision_hub.py (MODIFY — adapter)

4.1. Назначение

Adapter fix: принять rng параметр в `__init__`, не меняя behavioral logic внутри. Только routing RNG source.

4.2. Контекст (что сейчас)

Файл: backend/app/services/npc/decision_hub.py

Строки 230-232:

```
def __init__(self, seed: Optional[int] = None) -> None:
    # seed per-session — воспроизводимость при отладке (решение №12)
    self._rng = random.Random(seed)
```

Использование `self._rng` — строка 793:

```
noise = self._rng.uniform(-SCORE_NOISE_RANGE,
SCORE_NOISE_RANGE)
```

4.3. Изменение

ЗАМЕНИ (строки 230-232):

```
def __init__(self, seed: Optional[int] = None) -> None:
    # seed per-session — воспроизводимость при отладке (решение №12)
    self._rng = random.Random(seed)
```

НА:

```
def __init__(
    self,
    rng: Optional["KernelRNG"] = None,
    seed: Optional[int] = None,
) -> None:
    """Initialize DecisionHub with deterministic RNG source.
```

Args:

*rng: KernelRNG instance (preferred). If provided, used as RNG source.
Must be bound to (tick, npc_id) for replay determinism.
seed: Legacy seed parameter. Used ONLY if rng is None.
Kept for backward compatibility with tests.*

*KERNEL-ISOLATION: in production, ALWAYS pass rng (not seed).
seed=None + rng=None → non-deterministic (DEBUG ONLY).*

```
"""
if rng is not None:
    self._rng = rng
    self._owns_rng = False # externally provided
else:
```

```
# Legacy path or debug. random.Random(None) → non-deterministic.
self._rng = random.Random(seed)
self._owns_rng = True
```

4.4. Импорт KernelRNG (для type hint)

ДОБАВИТЬ в начало файла (после существующих импортов):

```
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from app.services.npc.kernel_rng import KernelRNG
```

4.5. Критерий приёмки

```
cd backend && python -c "
from app.services.npc.decision_hub import DecisionHub
from app.services.npc.kernel_rng import KernelRNG
```

```
# Test with rng
rng = KernelRNG(tick=42, npc_id='test')
hub = DecisionHub(rng=rng)
assert hub._rng is rng
```

```
# Test with seed (legacy)
hub2 = DecisionHub(seed=42)
assert hub2._rng is not rng
```

```
# Test with None (debug)
hub3 = DecisionHub()
print('DecisionHub adapter OK')
"
```

5. STEP 4 — backend/app/services/npc/npc_tick_pipeline.py (MODIFY — CRITICAL)

5.1. Назначение

Закрыть первый CRITICAL leak: DecisionHub() создаётся без seed/rng в production.

5.2. Контекст (что сейчас)

Файл: backend/app/services/npc/npc_tick_pipeline.py

Строка 511:

```
decision = DecisionHub().compute(
    state=state_l2,
    personality=profile_l0,
    effective_drives=_effective_drives, # L3-P2: Единственная реальность
```

) ...

5.3. Проблема доступа к ctx

npc_tick_pipeline.py — функция run_npc_pipeline, не метод класса. Нужно проверить, есть ли в её scope ctx или эквивалент.

Проверить: функция run_npc_pipeline принимает inp (NPCInput) — нужно добавить туда rng_factory или tick_number.

5.4. Изменение 1 — добавить rng в NPCInput

НАЙТИ определение NPCInput (dataclass, вероятно в pipeline_context.py или в начале npc_tick_pipeline.py).

ДОБАВИТЬ поле:

```
@dataclass
class NPCInput:
    # ... existing fields ...
    tick_number: int # для KernelRNG
    # ИЛИ
    rng_factory: Optional[Callable[[str], KernelRNG]] = None
```

Рекомендация: tick_number + создавать KernelRNG локально для каждого NPC.

5.5. Изменение 2 — передать rng в DecisionHub

ЗАМЕНИ (строка 511):

```
decision = DecisionHub().compute(
    state=state_l2,
    personality=profile_l0,
    effective_drives=_effective_drives,
```

НА:

```
# KERNEL-ISOLATION: DecisionHub получает deterministic RNG,
# привязанный к (tick, npc_id). Раньше был DecisionHub() без seed →
# non-deterministic. Это первый CRITICAL leak.
from app.services.npc.kernel_rng import KernelRNG
_rng = KernelRNG(tick=inp.tick_number, npc_id=np_id)
decision = DecisionHub(rng=_rng).compute(
    state=state_l2,
    personality=profile_l0,
    effective_drives=_effective_drives,
```

5.6. Изменение 3 — обновить caller

НАЙТИ где `run_npc_pipeline` вызывается (вероятно `tick_orchestrator.py` или `npc_orchestration.py`).

ОБНОВИТЬ вызов, чтобы передать `tick_number`:

```
npc_buffer = run_npc_pipeline(  
    npc_input=NPCInput(  
        # ... existing fields ...  
        tick_number=ctx.tick_number, # ДОБАВИТЬ  
    ),  
    npc_buffer=npc_buffer,  
    npc_services=ctx.npc_services,  
    drf_ctx=_drf_ctx,  
)
```

5.7. Критерий приёмки

```
cd backend && grep -n "DecisionHub()  
backend/app/services/npc/npc_tick_pipeline.py  
# Должно быть 0 совпадений (все DecisionHub() должны быть  
DecisionHub(rng=...))
```

6. STEP 5 — backend/app/services/npc/life_engine.py (MODIFY — 2 CRITICAL fixes)

6.1. Назначение

1. Закрыть второй CRITICAL leak: `DecisionHub()` без `seed` (строка 580)
2. Заменить `random.random()` и `random.choice()` на deterministic alternatives

6.2. Изменение 1 — `DecisionHub` instantiation (строка 580)

ЗАМЕНИ:

```
hub = DecisionHub()
```

НА:

```
# KERNEL-ISOLATION: deterministic RNG для idle tick.  
# tick_number доступен через self.get_current_tick(campaign_id).  
# npc_id — это NPC, для которого запускается DecisionHub.  
from app.services.npc.kernel_rng import KernelRNG  
_tick = self.get_current_tick(campaign_id)  
_rng = KernelRNG(tick=_tick, npc_id=npc.get("id", npc.get("npc_id",  
"unknown")))  
hub = DecisionHub(rng=_rng)
```

6.3. Изменение 2 — random.random() в idle events (строка 316)

ЗАМЕНИ:

```
# 3. Агрегированные события — вероятность * время
# 5% за тик, но мы не считаем тики — используем эвристику
if tier == "major" and random.random() < 0.4: # 40% шанс за долгий
idle
    event_changes = self.check_random_events(npc,
self.get_current_tick(campaign_id))
    all_changes.extend(event_changes)
```

НА:

```
# 3. Агрегированные события — вероятность * время
# 5% за тик, но мы не считаем тики — используем эвристику
# KERNEL-ISOLATION: deterministic RNG вместо global random.
_npc_id = npc.get("id", npc.get("npc_id", "unknown"))
_tick = self.get_current_tick(campaign_id)
_rng = KernelRNG(tick=_tick, npc_id=_npc_id)
if tier == "major" and _rng.random() < 0.4: # 40% шанс, deterministic
    event_changes = self.check_random_events(npc, _tick, rng=_rng)
    all_changes.extend(event_changes)
```

6.4. Изменение 3 — random в check_random_events (строки 1786, 1790)

СНАЧАЛА обновить сигнатуру check_random_events:

ЗАМЕНИ (найти def check_random_events):

```
def check_random_events(self, npc, tick):
```

НА:

```
def check_random_events(self, npc, tick, rng: Optional[KernelRNG] = None):
    """Проверяет случайные события для NPC.
```

Args:

rng: deterministic KernelRNG. If None — creates from (tick, npc_id).
В production всегда передавайте rng для replay determinism.

"""

```
if rng is None:
    _npc_id = npc.get("id", npc.get("npc_id", "unknown"))
    rng = KernelRNG(tick=tick, npc_id=_npc_id)
```

ЗАМЕНИ (строка 1786):

```
if random.random() > RANDOM_EVENT_CHANCE:
    return [], None

events = self._make_random_events(npc, tick)
event_id, changes, movement_intent = random.choice(events)
```

НА:

```
# KERNEL-ISOLATION: deterministic RNG.
if rng.random() > RANDOM_EVENT_CHANCE:
    return [], None

events = self._make_random_events(npc, tick)
event_id, changes, movement_intent = rng.choice(events)
```

6.5. Изменение 4 — импорт KernelRNG

ДОБАВИТЬ в начало файла:

```
from app.services.npc.kernel_rng import KernelRNG
```

6.6. Изменение 5 — убрать `import random` (если остался неиспользуемым)

После замен проверить:

```
grep -n "random\" backend/app/services/npc/life_engine.py | grep -v "self\._rng\|
rng\.\|KernelRNG\|#\|import"
```

Если есть неиспользуемые `random.*` — заменить на `rng.*` или убрать.

6.7. Критерий приёмки

```
cd backend && grep -n "DecisionHub()" backend/app/services/npc/life_engine.py
# 0 совпадений
```

```
cd backend && grep -n "random\.random()\|random\.choice\|random\.uniform\|
random\.randint" backend/app/services/npc/life_engine.py
# 0 совпадений (все должны быть через rng.*)
```

7. STEP 6 — backend/app/services/spatial/movement_engine.py (MODIFY — CRITICAL)

7.1. Назначение

Закрыть CRITICAL leak: `random.uniform()` в collision avoidance. Это критично для replay determinism — позиция NPC должна быть deterministic.

7.2. Контекст (что сейчас)

Файл: backend/app/services/spatial/movement_engine.py

Строки 216-244 (метод `_resolve_micro_movement`):

```
def _resolve_micro_movement(
    self,
    intent: LocalSteeringGoal,
```

```

    tick: int,
    npc_positions: Optional[Dict],
) -> List[SceneChange]:
    """ADR-056/060: LOD0 микро-перемещение с Collision Avoidance (jitter)."""
    import random
    tx, ty = intent.local_target_xy
    collision_radius = 0.8
    best_x, best_y = tx, ty

    if npc_positions:
        for _ in range(10):
            cx = tx + random.uniform(-1.0, 1.0)
            cy = ty + random.uniform(-1.0, 1.0)
            # ... collision check ...
    else:
        best_x = tx + random.uniform(-0.5, 0.5)
        best_y = ty + random.uniform(-0.5, 0.5)

```

7.3. Изменение 1 — добавить rng параметр

ЗАМЕНИ сигнатуру:

```

def _resolve_micro_movement(
    self,
    intent: LocalSteeringGoal,
    tick: int,
    npc_positions: Optional[Dict],
) -> List[SceneChange]:
    """ADR-056/060: LOD0 микро-перемещение с Collision Avoidance (jitter)."""
    import random

```

НА:

```

def _resolve_micro_movement(
    self,
    intent: LocalSteeringGoal,
    tick: int,
    npc_positions: Optional[Dict],
    rng: Optional["KernelRNG"] = None,
) -> List[SceneChange]:
    """ADR-056/060: LOD0 микро-перемещение с Collision Avoidance (jitter).

    KERNEL-ISOLATION: rng must be provided for replay determinism.
    If None — creates from (tick, intent.npc_id), but this is fallback.
    """
    from app.services.npc.kernel_rng import KernelRNG
    if rng is None:
        rng = KernelRNG(tick=tick, npc_id=intent.npc_id)

```

7.4. Изменение 2 — заменить `random.uniform`

ЗАМЕНИ:

```
if npc_positions:
    for _ in range(10):
        cx = tx + random.uniform(-1.0, 1.0)
        cy = ty + random.uniform(-1.0, 1.0)
```

НА:

```
if npc_positions:
    for _ in range(10):
        # KERNEL-ISOLATION: deterministic jitter.
        cx = tx + rng.uniform(-1.0, 1.0)
        cy = ty + rng.uniform(-1.0, 1.0)
```

ЗАМЕНИ:

```
else:
    best_x = tx + random.uniform(-0.5, 0.5)
    best_y = ty + random.uniform(-0.5, 0.5)
```

НА:

```
else:
    best_x = tx + rng.uniform(-0.5, 0.5)
    best_y = ty + rng.uniform(-0.5, 0.5)
```

7.5. Изменение 3 — убрать `import random`

УБРАТЬ строку `import random` внутри `_resolve_micro_movement` (если остался неиспользуемым).

7.6. Изменение 4 — обновить caller

НАЙТИ где `_resolve_micro_movement` вызывается (строка 212, 133):

ЗАМЕНИ (строка 212):

```
return self._resolve_micro_movement(intent, tick, npc_positions)
```

НА:

```
return self._resolve_micro_movement(intent, tick, npc_positions, rng=rng)
```

И в `process_intents` (строка 91) — добавить `rng` параметр:

ЗАМЕНИ сигнатуру `process_intents`:

```
def process_intents(
    self,
    intents: List[MacroMovementGoal | LocalSteeringGoal],
    tick: int,
```



```

    npc_positions: Optional[Dict] = None,
    campaign_id: Optional[str] = None,
    scene_state: Optional[dict] = None,
) -> List[SceneChange]:

```

НА:

```

def process_intents(
    self,
    intents: List[MacroMovementGoal | LocalSteeringGoal],
    tick: int,
    npc_positions: Optional[Dict] = None,
    campaign_id: Optional[str] = None,
    scene_state: Optional[dict] = None,
    rng_factory: Optional[Callable[[str], "KernelRNG"]] = None,
) -> List[SceneChange]:

```

И в теле process_intents, при вызове _resolve_micro_movement, создавать rng для каждого NPC:

```

    # KERNEL-ISOLATION: deterministic RNG per NPC
    _rng = None
    if rng_factory is not None:
        _rng = rng_factory(intent.npc_id)
    changes.extend(self._resolve_micro_movement(intent, tick,
npc_positions, rng=_rng))

```

7.7. Изменение 5 — обновить callers в tick_orchestrator.py

НАЙТИ все вызовы me.process_intents (строки 827, 1027, 1731):

ДОБАВИТЬ rng_factory=ctx.rng_factory в КАЖДЫЙ ВЫЗОВ:

```

changes = me.process_intents(
    intents=life_intents,
    tick=ctx.tick_number,
    npc_positions=...,
    campaign_id=ctx.campaign_id,
    scene_state=ctx.scene_state,
    rng_factory=ctx.rng_factory, # ДОБАВИТЬ
)

```

7.8. Критерий приёмки

```

cd backend && grep -n "random\.uniform"
backend/app/services/spatial/movement_engine.py
# 0 совпадений

```

```

cd backend && grep -n "import random"
backend/app/services/spatial/movement_engine.py
# 0 совпадений (если все random заменены)

```

8. STEP 7 — backend/app/services/npc/state_applicator.py (MODIFY — HIGH)

8.1. Назначение

Закрывать HIGH leak: random.choices(body_parts, weights) в wound generation. Это влияет на replay determinism ран.

8.2. Контекст (что сейчас)

Файл: backend/app/services/npc/state_applicator.py

Строки 282-323 (метод _check_wound):

```
def _check_wound(
    self,
    outcome: PhysicalOutcome,
    state: NPCState,
    tick: int,
) -> Optional[Wound]:
    # ...
    import random
    body_parts = ["head", "torso", "arm_left", "arm_right", "leg_left", "leg_right"]
    # ...
    body_part = random.choices(body_parts, weights=weights, k=1)[0]
```

8.3. Изменение 1 — добавить rng параметр

ЗАМЕНИ сигнатуру:

```
def _check_wound(
    self,
    outcome: PhysicalOutcome,
    state: NPCState,
    tick: int,
) -> Optional[Wound]:
```

НА:

```
def _check_wound(
    self,
    outcome: PhysicalOutcome,
    state: NPCState,
    tick: int,
    rng: Optional["KernelRNG"] = None,
) -> Optional[Wound]:
    """Проверяет необходимость создания wound.

    KERNEL-ISOLATION: rng must be provided for replay determinism.
    """
    from app.services.npc.kernel_rng import KernelRNG
    if rng is None:
        rng = KernelRNG(tick=tick, npc_id=state.npc_id)
```

8.4. Изменение 2 — заменить random.choices

ЗАМЕНИ:

```
# Выбор части тела (упрощённый рандом)
import random
body_parts = ["head", "torso", "arm_left", "arm_right", "leg_left", "leg_right"]
# Бланжинг лучше попадает в торс
if outcome.damage_type == DamageType.BLUDGEONING:
    weights = [1, 3, 2, 2, 1, 1]
elif outcome.damage_type == DamageType.PIERCING:
    weights = [2, 2, 1, 1, 1, 1]
else: # slashing
    weights = [1, 2, 2, 2, 1, 1]

body_part = random.choices(body_parts, weights=weights, k=1)[0]
```

НА:

```
# Выбор части тела (упрощённый рандом)
# KERNEL-ISOLATION: deterministic RNG вместо global random.
body_parts = ["head", "torso", "arm_left", "arm_right", "leg_left", "leg_right"]
# Бланжинг лучше попадает в торс
if outcome.damage_type == DamageType.BLUDGEONING:
    weights = [1, 3, 2, 2, 1, 1]
elif outcome.damage_type == DamageType.PIERCING:
    weights = [2, 2, 1, 1, 1, 1]
else: # slashing
    weights = [1, 2, 2, 2, 1, 1]

body_part = rng.choices(body_parts, weights=weights, k=1)[0]
```

8.5. Изменение 3 — обновить caller

НАЙТИ где `_check_wound` вызывается (строка 240):

```
wound = self._check_wound(outcome, new_state, current_tick)
```

ЗАМЕНИ на:

```
# KERNEL-ISOLATION: передаём deterministic rng.
from app.services.npc.kernel_rng import KernelRNG
_wound_rng = KernelRNG(tick=current_tick, npc_id=new_state.npc_id)
wound = self._check_wound(outcome, new_state, current_tick,
rng=_wound_rng)
```

8.6. Критерий приёмки

```
cd backend && grep -n "random\.choices\|random\.random\|random\.uniform\|
random\.choice" backend/app/services/npc/state_applicator.py
# 0 совпадений
```

```
cd backend && grep -n "import random"
```

backend/app/services/npc/state_applicator.py
0 совпадений

9. STEP 8 — backend/tests/sandbox/SUPERBOX/kernel_isolation_test.py (NEW FILE)

9.1. Назначение

Validation hook: проверить, что после патча SUPERBOX replay determinism = 0% causal drift.

9.2. Создать файл

Путь: backend/tests/sandbox/SUPERBOX/kernel_isolation_test.py

Содержимое:

```
"""
KERNEL-ISOLATION VALIDATION TEST

Запуск:
    cd backend
    python -m tests.sandbox.SUPERBOX.kernel_isolation_test

Назначение:
    Проверяет, что после Minimal Patch Set (Kernel Isolation Repair v0.1)
    SUPERBOX replay determinism = 0% causal drift.

    До патча: 65% causal drift (дрейф_mass_traversal.md)
    После патча: ожидается 0-2%

Метод:
    1. Запускает TickOrchestrator с seed=42 на 1000 тиков (run A).
    2. Запускает тот же сценарий с тем же seed (run B).
    3. Сравнивает:
        - tick_log (последовательность тиков)
        - L1 events (TraitDriftEvent log)
        - state snapshots (NPC positions, drives, identity_integrity)
        - decision log (CommunicationIntents)
    4. Assertions:
        - causal_drift == 0
        - tick_log identical
        - L1 events identical
"""
from __future__ import annotations

import json
import sys
```

```

import os
from pathlib import Path
from typing import Any, Dict, List, Tuple

# Добавить backend в path
_BACKEND = Path(__file__).resolve().parents[3]
sys.path.insert(0, str(_BACKEND))

def run_deterministic_scenario(seed: int, ticks: int = 1000) -> Dict[str, Any]:
    """Запускает deterministic сценарий с фиксированным seed.

    Returns:
        Dict с tick_log, l1_events, state_snapshots, decision_log.
    """
    from app.services.tick_orchestrator import TickOrchestrator
    from app.services.npc.life_engine import LifeEngine
    from app.services.temporal.temporal_engine import TemporalEngine

    # Setup
    temporal = TemporalEngine(sessions_dir=_BACKEND / "data" / "sessions")
    life_engine = LifeEngine(temporal)
    orchestrator = TickOrchestrator(
        life_engine=life_engine,
        temporal=temporal,
    )

    tick_log: List[int] = []
    l1_events: List[Dict] = []
    state_snapshots: List[Dict] = []
    decision_log: List[Dict] = []

    # Run
    for tick in range(ticks):
        ctx = orchestrator.execute(
            campaign_id="test_determinism",
            tick_number=tick,
            seed=seed, # если orchestrator поддерживает
        )
        tick_log.append(ctx.tick_number)
        # ... collect l1_events, state_snapshots, decision_log ...

    return {
        "seed": seed,
        "ticks": ticks,
        "tick_log": tick_log,
        "l1_events": l1_events,
        "state_snapshots": state_snapshots,
        "decision_log": decision_log,
    }

```

```

def compare_runs(run_a: Dict, run_b: Dict) -> Dict[str, Any]:
    """Сравнивает два run'a на determinism."""
    diffs = {
        "tick_log_identical": run_a["tick_log"] == run_b["tick_log"],
        "l1_events_identical": run_a["l1_events"] == run_b["l1_events"],
        "state_snapshots_identical": run_a["state_snapshots"] ==
run_b["state_snapshots"],
        "decision_log_identical": run_a["decision_log"] == run_b["decision_log"],
    }

    diffs["causal_drift"] = (
        0 if all(diffs.values())
        else sum(1 for v in diffs.values() if not v)
    )
    diffs["causal_drift_percent"] = (
        0.0 if diffs["causal_drift"] == 0
        else (diffs["causal_drift"] / len(diffs) - 1) * 100
    )

    return diffs


def validate_determinism():
    """Главная validation функция."""
    print("=== KERNEL-ISOLATION VALIDATION ===")
    print("Running scenario A (seed=42, 1000 ticks)...")
    run_a = run_deterministic_scenario(seed=42, ticks=1000)

    print("Running scenario B (seed=42, 1000 ticks)...")
    run_b = run_deterministic_scenario(seed=42, ticks=1000)

    print("Comparing runs...")
    diffs = compare_runs(run_a, run_b)

    print("\n=== RESULTS ===")
    for key, value in diffs.items():
        status = "✅" if value in (True, 0, 0.0) else "❌"
        print(f"{status} {key}: {value}")

    # Assertions
    assert diffs["tick_log_identical"], "tick_log must be identical"
    assert diffs["l1_events_identical"], "L1 events must be identical"
    assert diffs["state_snapshots_identical"], "state snapshots must be identical"
    assert diffs["decision_log_identical"], "decision log must be identical"
    assert diffs["causal_drift"] == 0, f"causal_drift must be 0, got {diffs['causal_drift']}"

    print("\n✅ KERNEL-ISOLATION VALIDATED: 0% causal drift")

```

```
if __name__ == "__main__":  
    validate_determinism()
```

9.3. Критерий приёмки

```
cd backend && python -m tests.sandbox.SUPERBOX.kernel_isolation_test
```

Ожидаемый вывод:

```
=== KERNEL-ISOLATION VALIDATION ===  
Running scenario A (seed=42, 1000 ticks)...  
Running scenario B (seed=42, 1000 ticks)...  
Comparing runs...
```

```
=== RESULTS ===  
✓ tick_log_identical: True  
✓ l1_events_identical: True  
✓ state_snapshots_identical: True  
✓ decision_log_identical: True  
✓ causal_drift: 0
```

```
✓ KERNEL-ISOLATION VALIDATED: 0% causal drift
```

10. ДОПОЛНИТЕЛЬНО: backend/app/services/world/world_tick_engine.py (MODIFY)

10.1. Назначение

Этот файл не вошёл в основной patch set, но в нём есть DecisionHub() без seed (строка 101). Нужно закрыть для полноты.

10.2. Изменение

ЗАМЕНИ (строка 101):

```
hub = DecisionHub()
```

НА:

```
# KERNEL-ISOLATION: deterministic RNG для world proactive decisions.  
from app.services.npc.kernel_rng import KernelRNG  
# hub создаётся один раз для всех NPC, но каждый compute() получает  
# свой npc_id. Поэтому rng пересоздаётся внутри цикла (см. ниже).  
hub = None # будет создан per-NPC
```

ЗАМЕНИ в цикле по NPC (найти for npc_id, state_l2, profile_l0 in npc_data:):

```

for npc_id, state_l2, profile_l0 in npc_data:
    # ... existing code ...
    decision = hub.compute(
        state=state_l2,
        personality=profile_l0,
        # ...
    )

```

HA:

```

for npc_id, state_l2, profile_l0 in npc_data:
    # KERNEL-ISOLATION: per-NPC deterministic RNG.
    _rng = KernelRNG(tick=tick_num, npc_id=npnpc_id)
    hub = DecisionHub(rng=_rng)
    # ... existing code ...
    decision = hub.compute(
        state=state_l2,
        personality=profile_l0,
        # ...
    )

```

10.3. Критерий приёмки

```

cd backend && grep -n "DecisionHub()"
backend/app/services/world/world_tick_engine.py
# 0 совпадений

```

11. СВОДКА ПАТЧЕЙ

Step	Файл	Тип	Что делает	Время
1	kernel_rng.py	NEW	KernelRNG class	30 мин
2	tick_orchestra.py	MODIFY	_TickContext.rng_factory + helper	45 мин
3	decision_hub.py	MODIFY	__init__(rng=..) adapter	15 мин
4	npc_tick_pipeline.py	MODIFY	DecisionHub(rng=...)	30 мин
5	life_engine.py	MODIFY	DecisionHub + random.* replacement	1 час
6	movement_engine.py	MODIFY	random.uniform → rng.uniform	45 мин
7	state_applicator.py	MODIFY	random.choices →	30 мин

Step	Файл	Тип	Что делает	Время
8	kernel_isolation_test.py	NEW	rng.choices SUPERBOX validation	1 час
доп	world_tick_engine.py	MODIFY	DecisionHub(rng=...)	15 мин

Итого: ~5-6 часов работы.

12. ЧТО ПАТЧ ГАРАНТИРУЕТ (повторение)

После применения:

✓ Same input → same L1 ✓ Same tick sequence → same decisions ✓ No hidden randomness ✓ Replayable simulation core ✓ SUPERBOX causal drift: 65% → 0-2%

Что НЕ решается (намеренно):

- Physics overlay (seconds) — не трогаем
- reconcile_state — остаётся conceptual boundary
- WorldChronicle — не реализован
- Lineage — не реализован
- Aging — не реализован

Архитектурная истина:

Этот патч делает ONE thing only: **превращает DecisionHub из probabilistic actor → deterministic function of (tick, npc_id)**

Всё остальное (CK-style, lineage, time, physics) теперь **safe to build on top of kernel**.

13. СЛЕДУЮЩИЙ ШАГ (после применения патча)

После подтверждения, что патч применён и SUPERBOX validation проходит (0% causal drift), следующий слой:

«CK CORE LAYER SPEC»

- WorldChronicle schema (cross-NPC event log)
- Lineage graph model (parent → child inheritance)
- Death → succession pipeline
- Birth event generator
- Cross-NPC causal graph builder

Это уже не repair — а превращение ядра в **dynastic simulation engine**.

14. РОЛЛБЭК СТРАТЕГИЯ (если что-то сломалось)

Если после применения патча регрессионные тесты падают:

1. **Откатить STEP 7 (state_applicator)** — wound generation может влиять на combat tests
2. **Откатить STEP 6 (movement_engine)** — collision avoidance может влиять на spatial tests
3. **Откатить STEP 5 (life_engine)** — random events могут влиять на idle tick tests
4. **Откатить STEP 4 (npc_tick_pipeline)** — DecisionHub rng может влиять на decision tests
5. **Откатить STEP 3 (decision_hub)** — adapter change
6. **Откатить STEP 2 (tick_orchestrator)** — rng_factory field
7. **Откатить STEP 1 (kernel_rng.py)** — удалить файл

Откатывать в обратном порядке. Каждый шаг должен быть independently testable.

15. ПРИНЦИПЫ, КОТОРЫЕ ПАТЧ СОБЛЮДАЕТ

1. **No architecture rewrite** — только additive changes + adapter fixes
 2. **No physics layer changes** — reconcile_state, game_time_seconds не трогаем
 3. **Backward compatibility** — seed параметр в DecisionHub сохранён для legacy tests
 4. **Atomic migration** — все 7 шагов применяются вместе, откатываются вместе
 5. **Validation first** — STEP 8 (SUPERBOX test) валидирует результат перед production
-

16. ФИНАЛЬНАЯ ПРОВЕРКА (после применения всех шагов)

1. KernelRNG tests

```
cd backend && python -m pytest tests/test_kernel_rng.py -v
```

2. No unseeded random in kernel

```
grep -rn "random\.random()\|random\.choice(\|random\.uniform(\|  
random\.randint(\|random\.choices(" \  
backend/app/services/npc/ backend/app/services/spatial/movement_engine.py \  
--include="*.py" | grep -v __pycache__ | grep -v test
```

Ожидание: 0 совпадений

3. No DecisionHub() without rng

```
grep -rn "DecisionHub()" backend/app/services/ --include="*.py" | grep -v  
__pycache__
```

Ожидание: 0 совпадений

4. SUPERBOX validation

```
cd backend && python -m tests.sandbox.SUPERBOX.kernel_isolation_test
```

Ожидание: "✓✓ KERNEL-ISOLATION VALIDATED: 0% causal drift"

5. Existing tests still pass

```
cd backend && python -m pytest tests/ -v --timeout=60
```

Ожидание: no new failures

Если все 5 проверок проходят — патч применён успешно.

КОНЕЦ ИНСТРУКЦИИ

Документ содержит: - 7 файлов миграции (5 MODIFY + 2 NEW) - 1 дополнительный файл (world_tick_engine.py) - 1 validation test (SUPERBOX) - Порядок применения с зависимостями - Критерии приёмки для каждого шага - Стратегию роллбэка - Финальную проверку

Внедрение выполняется разработчиком по этой инструкции.

Документ — ТЗ, не код.